LOVELY PROFESSIONAL UNIVERSITY

Mittal School of Business

PREDICTIVE ANALYTICS

(INTM591)

Project Report

Course Code: INTM591	**Course Title: Predictive Analytics**
Name: Abhijith Krishnan P S	**Academic Task Title: Project**
Academic Task No: 01	**Student Reg. No:12512848**
Student Roll No: RQ5E09B40	**Section: Q5E09**

Exploratory Data Analysis of FIFA Dataset using Python

Working:

Python is utilized for its powerful data manipulation and visualization capabilities. In this analysis, we perform data cleaning, statistical summarization, and visualization to understand player demographics, physical attributes, and performance patterns in the FIFA dataset.

Step 1: Data Loading and Initial Inspection

• We load the dataset and use `.head()` and `.info()` to understand the structure, data types, and identify missing values or inconsistencies.

• Load dataset

• Inspect data

Step 2: Data Cleaning and Preprocessing

• Missing values are identified and handled appropriately.

• Columns are cleaned and converted into proper data types for analysis.

• Numerical columns like Age and Height are ensured to be in correct format.

• Check missing values

• Handle missing values

• Convert data types if needed

Step 3: Age Distribution (Pattern Exploration)

• A histogram is plotted to understand how player ages are distributed across the dataset.

• This helps identify the most common age group of players.

• Visualization of Data.

• Observation:

Most players fall between 20–30 years, indicating peak performance age in professional football.

Step 4: Outlier Detection using Boxplot (Age)

• A boxplot is used to detect outliers in player age.

• This helps identify unusually young or old players.

• Observation:

Median age ≈ 25 years

Outliers above 38–45 years, representing older players

Step 5: Height Distribution and Outlier Analysis

• A boxplot is used to analyze player height distribution and detect extreme values.

• Observation:

Most players fall between 5.7 and 6.2 feet

Outliers represent very short and very tall players

Step 6: Insights and Interpretation

• The majority of players are concentrated in the 20–30 age range, which aligns with peak athletic performance.

• Height distribution is relatively consistent, indicating standard physical requirements in football.

• Outliers highlight unique player profiles (e.g., very tall goalkeepers or experienced senior players).

Predictive Analysis using Machine Learning

Working:

Machine learning techniques are applied to predict player wages based on key attributes such as age, overall rating, and potential. A regression model is used to identify relationships between features and financial outcomes.

Step 1: Feature Selection

• Relevant features are selected for prediction: Age, Overall Rating, Potential

• Target variable: Wage

Step 2: Train-Test Split

• The dataset is split into training and testing sets to evaluate model performance.

Step 3: Model Building (Gradient Boosting Regressor)

• A Gradient Boosting Regressor is used for prediction due to its high accuracy in regression problems.

Step 4: Model Prediction

• The trained model is used to predict wages on test data.

Step 5: Model Evaluation

• Performance is evaluated using metrics like Mean Absolute Error (MAE) and R^2 score.

Step 6: Insights from Model

• Player wages are strongly influenced by: Overall rating, Potential. Age.

• Higher-rated players tend to have significantly higher wages.

Conclusion

The project successfully analyzes the FIFA dataset using Python to uncover patterns in player demographics and performance. The EDA shows that most players are in their peak age range and follow consistent physical characteristics. The machine learning model further demonstrates that player wages can be effectively predicted using key attributes like overall rating and potential. This analysis provides valuable insights into player valuation and can be extended for advanced scouting, recruitment strategies, and performance prediction.

Screenshots

```
#Exploratory Data Analysis
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("/content/drive/MyDrive/fifa dataset.csv")
df.head()
```

	ID	Name	Age	Nationality	Overall	Potential	Club	Value	Wage	Preferred Foot	International Reputation	Skill Moves	Position	Joined	Contract Valid Until	Height	Weight	Release Clause
0	158023	L. Messi	31	Argentina	94	94	FC Barcelona	110500.0	565.0	Left	5.0	4.0	RF	2004	2021-01-01	5.583333	159.0	226500.0
1	20801	Cristiano Ronaldo	33	Portugal	94	94	Juventus	77000.0	405.0	Right	5.0	5.0	ST	2018	2022-01-01	6.166667	183.0	127100.0
2	190871	Neymar Jr	26	Brazil	92	93	Paris Saint-Germain	118500.0	290.0	Right	5.0	5.0	LW	2017	2022-01-01	5.750000	150.0	228100.0
3	193080	De Gea	27	Spain	91	93	Manchester United	72000.0	260.0	Right	4.0	1.0	GK	2011	2020-01-01	6.333333	168.0	138600.0
4	192985	K. De Bruyne	27	Belgium	91	92	Manchester City	102000.0	355.0	Right	4.0	4.0	RCM	2015	2023-01-01	5.916667	154.0	196400.0

```
df.isnull().sum()
```

	0
ID	0
Name	0
Age	0
Nationality	0
Overall	0
Potential	0
Club	241
Value	252
Wage	0
Preferred Foot	0
International Reputation	48
Skill Moves	48
Position	0
Joined	0
Contract Valid Until	289
Height	0

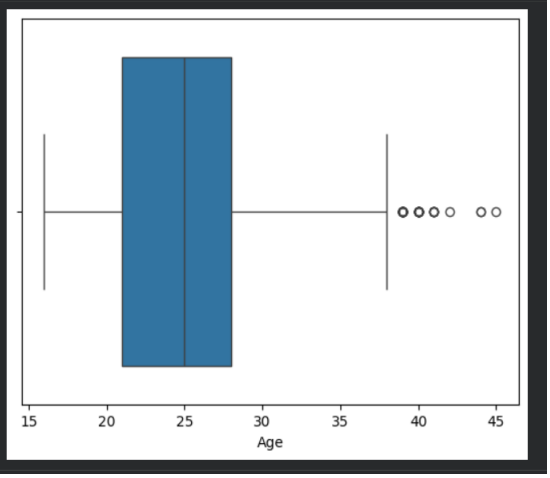
```
df.dtypes
```

	0
ID	int64
Name	object
Age	int64
Nationality	object
Overall	int64
Potential	int64
Club	object
Value	float64
Wage	float64
Preferred Foot	object
International Reputation	float64
Skill Moves	float64
Position	object
Joined	int64
Contract Valid Until	object
Height	float64

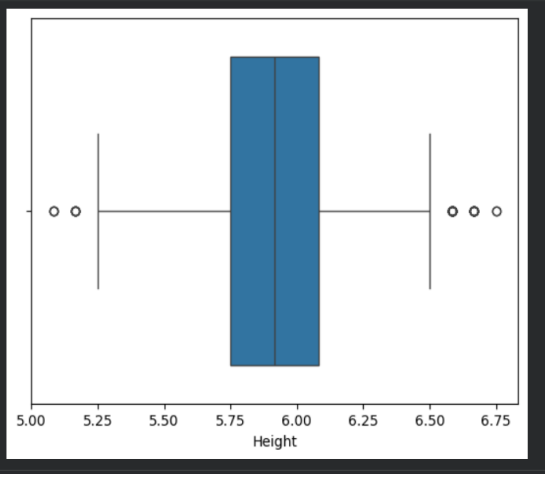
```
df.describe()
```

	ID	Age	Overall	Potential	Value	Wage	International Reputation	Skill Moves	Joined	Height	Weight	Release Clause
count	18207.000000	18207.000000	18207.000000	18207.000000	17955.000000	18207.000000	18159.000000	18159.000000	18207.000000	18207.000000	18207.000000	18207.000000
mean	214298.338606	25.122206	66.238699	71.307299	2444.530214	9.731312	1.113222	2.361308	2016.420607	5.946771	165.979129	4585.060971
std	29965.244204	4.669943	6.908930	6.136496	5626.715434	21.999290	0.394031	0.756164	2.018194	0.220514	15.572775	10630.414430
min	16.000000	16.000000	46.000000	48.000000	10.000000	0.000000	1.000000	1.000000	1991.000000	5.083333	110.000000	13.000000
25%	200315.500000	21.000000	62.000000	67.000000	325.000000	1.000000	1.000000	2.000000	2016.000000	5.750000	154.000000	570.000000
50%	221759.000000	25.000000	66.000000	71.000000	700.000000	3.000000	1.000000	2.000000	2017.000000	5.916667	165.000000	1300.000000
75%	236529.500000	28.000000	71.000000	75.000000	2100.000000	9.000000	1.000000	3.000000	2018.000000	6.083333	176.000000	4585.060806
max	246620.000000	45.000000	94.000000	95.000000	118500.000000	565.000000	5.000000	5.000000	2018.000000	6.750000	243.000000	228100.000000

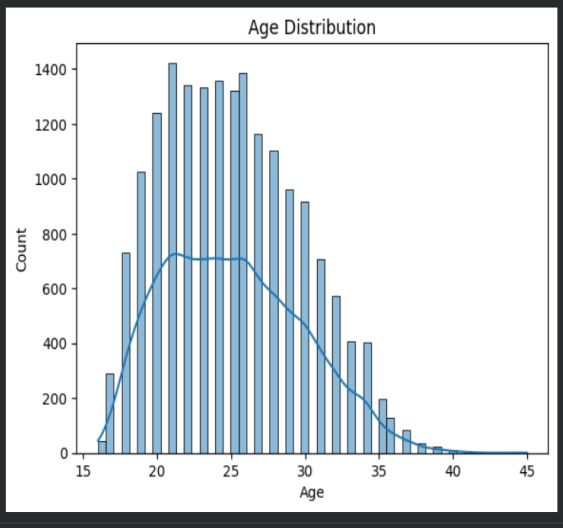
```python
sns.boxplot(x=df['Age'])
plt.show()
```



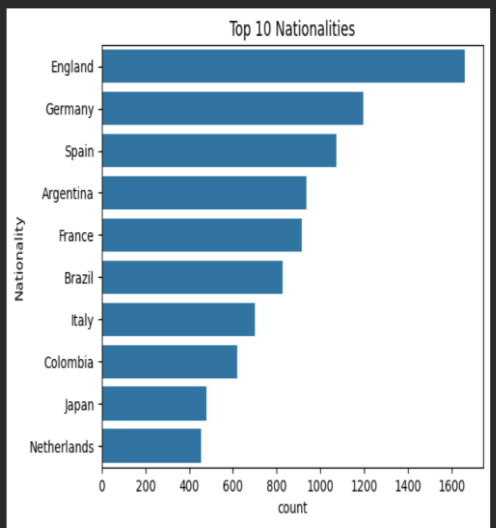

```python
sns.boxplot(x=df['Height'])
plt.show()
```




```python
sns.histplot(df['Age'], kde=True)
plt.title("Age Distribution")
plt.show()
```




```python
sns.countplot(y=df['Nationality'], order=df['Nationality'].value_counts().head(10).index)
plt.title("Top 10 Nationalities")
plt.show()
```




```python
features = ['Age','Club','International Reputation','Skill Moves',
            'Overall','Potential','Value','Height','Weight']

target = 'Wage'

X = df[features]
y = df[target]

# One hot encoding for categorical columns
X = pd.get_dummies(X, columns=['Club'], drop_first=True)
```

```python
X
```

	Age	International Reputation	Skill Moves	Overall	Potential	Value	Height	Weight	Club_1. FC Heidenheim 1846	Club_1. FC Kaiserslautern	...	Club_Wycombe Wanderers	Club_Yeni Malatyaspor	Club_Yeovil Town	Club_Yokohama F. Marinos	C
0	31	5.0	4.0	94	94	110500.0	5.583333	159.0	False	False	...	False	False	False	False	
1	33	5.0	5.0	94	94	77000.0	6.166667	183.0	False	False	...	False	False	False	False	
2	26	5.0	5.0	92	93	118500.0	5.750000	150.0	False	False	...	False	False	False	False	
3	27	4.0	1.0	91	93	72000.0	6.333333	168.0	False	False	...	False	False	False	False	
4	27	4.0	4.0	91	92	102000.0	5.916667	154.0	False	False	...	False	False	False	False	

```python
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.impute import SimpleImputer
import pandas as pd # Ensure pandas is available for DataFrame operations

# Impute missing values in X_train and X_test
# Using mean strategy for numerical data
imputer = SimpleImputer(strategy='mean')

# Fit imputer on X_train and transform both X_train and X_test
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)

# Convert back to DataFrame, keeping column names and index
X_train_imputed = pd.DataFrame(X_train_imputed, columns=X_train.columns, index=X_train.index)
X_test_imputed = pd.DataFrame(X_test_imputed, columns=X_test.columns, index=X_test.index)

gbr_model = GradientBoostingRegressor(n_estimators=200, random_state=42)
gbr_model.fit(X_train_imputed, y_train)
```

```
GradientBoostingRegressor
GradientBoostingRegressor(n_estimators=200, random_state=42)
```

```python
gbr_preds = gbr_model.predict(X_test_imputed)
```

```python
print("Gradient Boosting Regressor - Mean Absolute Error (MAE):", mean_absolute_error(y_test, gbr_preds))
print("Gradient Boosting Regressor - R2 Score:", r2_score(y_test, gbr_preds))
```

```
Gradient Boosting Regressor - Mean Absolute Error (MAE): 3.369132890062162
Gradient Boosting Regressor - R2 Score: 0.9131692176666499
```

```python
print("Gradient Boosting Regressor - Training Score:", gbr_model.score(X_train_imputed, y_train))
print("Gradient Boosting Regressor - Testing Score:", gbr_model.score(X_test_imputed, y_test))
```

```
Gradient Boosting Regressor - Training Score: 0.9219447777350528
Gradient Boosting Regressor - Testing Score: 0.9131692176666499
```